Herald

Protocol Research: Model Context Protocol (MCP)

Field	Value
Revision	1
Created	2026-05-22
Last modified	2026-05-22
Status	research-only (no implementation yet)
Status summary	MCP is the most mature LLM-to-tool integration protocol (Anthropic 2024-11; spec 2025-11-25; ecosystem-wide adoption). Herald should adopt MCP as its FIRST non-REST protocol because the use case maps directly: Herald exposes resources (tenant subscribers, events, safety state, compliance posture) + tools (notify, query) for LLM agents to consume. Official Go SDK exists, Apache 2.0, maintained in collaboration with Google.
Issues	open-questions: server-only vs server+client; resource granularity; OAuth 2.0 vs commons_auth-JWT layering.
Continuation	Wave 4a — open HRD-200..HRD-220 block; new module `commons_mcp/` (L1); every flavor gains `--mcp-stdio` + `--mcp-http` flags.

Constitutional anchors

- **§107 anti-bluff** — every MCP test below MUST produce physical proof (real client connecting to real server, byte-level response assertion). No metadata-only PASS.
- **§11.4.74 catalogue-check** — performed; no existing `vasic-digital` or `HelixDevelopment` MCP module found. Verdict: vendor `github.com/modelcontextprotocol/go-sdk` as a submodule under `submodules/go-mcp-sdk/` (Helix §3 no-`go get`-of-vendor rule) OR add as an exception per spec V3 §9.1.
- **§11.4.61 / branding** — this is a tracked doc; HTML/PDF/DOCX siblings to be regenerated via `scripts/export_docs.sh docs/research/protocols/MCP.md`.
- **Spec V3 implication** — Wave 4a will add a §4x.x to specification.V3.md describing the MCP surface. Per §106 spec-change rule, that spec edit triggers comprehensive planning + implementation in the same logical work effort.

Table of contents

§1. Protocol overview

MCP — Model Context Protocol. Open standard introduced by Anthropic in November 2024 to standardize how LLM applications integrate external data + tools. Now an open-source project with the spec at https://modelcontextprotocol.io, schemas under https://github.com/modelcontextprotocol/modelcontextprotocol, and language SDKs under https://github.com/modelcontextprotocol/ (TypeScript, Python, Go, Java, C#, Kotlin, Rust, Swift).

Author / Maintainer. Created by David Soria Parra + Justin Spahr-Summers at Anthropic. The Go SDK is maintained in collaboration with Google.

Current spec version (as of 2026-05-22): `2025-11-25`. The schema source-of-truth is `schema/2025-11-25/schema.ts` (TypeScript) with a derived JSON-Schema. The spec uses BCP-14 (RFC 2119) keywords (MUST / SHOULD / MAY).

Wire format. JSON-RPC 2.0 — every message is a JSON object with `jsonrpc: "2.0"`, an `id` (for requests/responses, omitted for notifications), `method`, and `params` (request) or `result`/`error` (response).

Transports.

- **stdio** — line-delimited JSON-RPC over stdin/stdout. Used for local integrations (LLM client launches the MCP server as a child process; Claude Desktop, Cursor, Continue, etc.).
- **Streamable HTTP** — REPLACED the deprecated HTTP+SSE transport in spec version 2025-03-26. Combines a single HTTP endpoint with optional server-pushed streaming. Spec: https://modelcontextprotocol.io/specification/2025-11-25/basic/transports.
- **SSE** (legacy) — DEPRECATED in 2025-03; retained only for backward compatibility. New implementations MUST use Streamable HTTP.

Authentication / authorization.

- stdio — process-level trust (parent LLM launched the server).
- Streamable HTTP — OAuth 2.0 client flow (RFC 6749) with dynamic client registration (RFC 7591) + the MCP `auth` extension (in spec since 2025-06). Capability tokens / signed access tokens are the recommended pattern.

Primary use cases.

1. LLM client (Claude Desktop, Cursor, Windsurf, Continue, Cody, …) calls into a tool server (PostgreSQL, GitHub, Slack, Stripe, Figma, …) to fetch context or invoke an operation. **200+ community servers** exist as of 2026.
2. LLM client samples (recursive LLM call) via the `sampling/createMessage` method — server-initiated, client-mediated.

3. Workflow assembly — `prompts/get` lets servers return reusable prompt templates.

Ecosystem adoption. First-class support in Claude Desktop, Cursor, Continue, Cody (Sourcegraph), Replit, Zed, Windsurf, Cline. Microsoft + GitHub adopted MCP for Copilot Workspace integration in early 2025. Public registry: https://github.com/modelcontextprotocol/servers.

Stability. The spec is dated by version (`YYYY-MM-DD`); breaking changes between versions are tracked in the `CHANGELOG`. The wire format (JSON-RPC 2.0) is rock-solid; the schema (resources/tools/prompts/sampling) has stabilized; the auth extension is the only active churn area as of 2026.

§2. Specification deep-dive

§2.1 Roles

- **Host** — the LLM application (Claude Desktop, Cursor, ...).
- **Client** — an MCP client instance INSIDE the host; manages one connection.
- **Server** — provides resources / tools / prompts.

A host can have many clients, each connected to one server.

§2.2 Lifecycle

1. **Initialize.** Client sends `initialize` request with declared `protocolVersion`, `capabilities`, `clientInfo`. Server responds with `protocolVersion`, `capabilities`, `serverInfo`.
2. **Initialized notification.** Client sends `initialized` notification — handshake complete.
3. **Operation.** Either side may send requests / notifications until shutdown.
4. **Shutdown.** Client closes the transport (stdio EOF / HTTP DELETE on session endpoint).

Example `initialize` (from spec §2.1):

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-11-25",
    "capabilities": {
      "roots": { "listChanged": true },
      "sampling": {}
    },
    "clientInfo": { "name": "claude-desktop", "version":
        "0.6.0" }
  }
}
```

§2.3 Server-offered features (the LLM-tool-server side)

- `resources` — context/data exposed to the host. Identified by URI; can have MIME types. Methods: `resources/list`, `resources/read`, `resources/subscribe`, `resources/unsubscribe`. Server notifies `notifications/resources/list_changed` and `notifications/resources/updated`.
- `prompts` — templated multi-turn prompts the host can offer to the user as workflows. Methods: `prompts/list`, `prompts/get`.
- `tools` — server-exposed functions the LLM can invoke. Each tool has a `name`, `description`, `inputSchema` (JSON Schema). Methods: `tools/list`, `tools/call`.
- **Logging** — `logging/setLevel` + `notifications/message` for log streaming.

§2.4 Client-offered features (the host side)

- `sampling` — server can ask the host to run an LLM inference (e.g. recursive agent). Method: `sampling/createMessage`. The user MUST explicitly approve (security principle #4).
- `roots` — server can query the host for filesystem/URI roots it's allowed to operate within. Method: `roots/list`.
- `elicitation` — server can request additional info from the user. (Newer feature, in spec since 2025-09.)

§2.5 Error model

Standard JSON-RPC 2.0 errors:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "code": -32602,
    "message": "Invalid params",
    "data": { "field": "uri", "reason": "must be absolute URI" }
  }
}
```

MCP defines a few protocol-specific codes in addition to the JSON-RPC reserved range (-32700 to -32099):

- `-32002` — Resource not found
- `-32001` — Server unavailable

§2.6 Capability negotiation + versioning

`protocolVersion` is the `YYYY-MM-DD` spec date. Client + server must agree; if not, the connection should be closed. Capabilities are declared at handshake; later messages MUST NOT use features the peer didn't declare.

§2.7 Streamable HTTP transport details

Replaces HTTP+SSE. Spec at `/specification/2025-11-25/basic/transports`. Key points:

- Single endpoint (e.g. `https://mcp.example.com/mcp`).
- Client POST JSON-RPC requests; server responds with either:
  - `Content-Type: application/json` (single response), or
  - `Content-Type: text/event-stream` (SSE-style streaming response — for tools that emit progress or multiple results).
- Session is established by the server returning an `Mcp-Session-Id` response header on `initialize`; subsequent requests carry it.
- Resumability: if a stream is interrupted, client reconnects with `Last-Event-ID` header (SSE-style ID resumption).
- Security: simpler than legacy SSE — no separate "out-of-band POST" channel that confused load balancers + firewalls. Sec: https://blog.fka.dev/blog/2025-06-06-why-mcp-deprecated-sse-and-go-with-streamable-http/.

§2.8 Security model

Per the spec §"Security and Trust & Safety":

1. **User Consent and Control** — explicit consent for data access + actions.
2. **Data Privacy** — host MUST obtain consent before exposing user data to servers.
3. **Tool Safety** — tools are arbitrary code execution; descriptions are untrusted unless from a trusted server.
4. **LLM Sampling Controls** — user MUST approve every sampling request; user controls visibility.

MCP itself cannot enforce these at protocol level — implementors MUST build consent flows.

§3. Herald-specific applicability analysis

§3.1 Which flavor binaries surface MCP?

Server side (Herald EXPOSES MCP servers — LLM agents call into Herald):

- **pherald** — exposes `events_processed` log as resources; exposes `notify` tool for outbound dispatch.
- **cherald** — exposes compliance posture (current attestations, controls, evidence) as resources; exposes `query_compliance` tool.
- **sherald** — exposes safety state (current memory pressure, OOM kills, recent incidents) as resources; exposes `query_safety` tool.
- **bherald** — exposes budget/billing state as resources.
- **rherald** — exposes risk register as resources.
- **iherald** — exposes incident timeline as resources; exposes `acknowledge_incident` tool.
- **scherald** — exposes schedule/cron state as resources.

Client side (Herald CONSUMES MCP servers — Herald talks to external MCP servers):

- All flavors via `commons_messaging/dispatch/mcp_client/` (new) — Herald uses MCP-client to invoke LLM tools (e.g. dispatch a "summarize this incident" prompt via a Claude / Anthropic MCP-aware host).

Recommendation: server-first. Server-side adoption is the highest payoff. Client-side is more useful as Herald's LLM dispatch surface matures.

§3.2 Does MCP replace or add to REST?

Adds. MCP is purpose-built for LLM-to-tool integration; REST remains the primary surface for HTTP clients (dashboards, CI/CD, webhooks, custom integrations). MCP is the LLM-facing **side channel** — same backend data, different transport semantics. Both surfaces share the same Postgres backend + audit log + idempotency machinery.

§3.3 Multi-tenant story

MCP has no native tenant concept. Herald MUST inject `tenant_id` at the transport edge:

- **stdio** — `pherald serve --mcp-stdio --tenant=<uuid>` binds one stdio MCP server to one tenant. The LLM client launches one pherald process per tenant. Simple but doesn't scale to many tenants.
- **Streamable HTTP** — `tenant_id` comes from the JWT in `Authorization: Bearer <jwt>` (the existing `commons_auth/` path). Single Herald deployment serves N tenants over one HTTP endpoint.

§3.4 JWT auth integration

Herald already has a JWT verifier in `commons_auth/`. Recommendation:

- **stdio** — out-of-band trust; no JWT needed (the parent process launched it).
- **Streamable HTTP** — MCP's OAuth 2.0 client flow is layered ON TOP of `commons_auth`:
  1. Client does OAuth dance against Herald's `/oauth/authorize` (Herald acts as OAuth Authorization Server — or delegates to Auth0 / Keycloak — TBD per operator).
  2. Client receives an access_token (a JWT signed by Herald's existing JWKS).
  3. Client passes `Authorization: Bearer <jwt>` on every MCP request.
  4. Herald's existing JWT middleware extracts `tenant_id` + `subject` claims.

Alternative (simpler MVP): skip OAuth dance; require pre-provisioned long-lived JWTs (matches Herald's current REST auth). Document the OAuth path as Wave 4a+1 enhancement.

§3.5 Idempotency

MCP `tools/call` requests have no native idempotency key. Herald injects one per-tool:

- `notify` tool — caller MUST pass `idempotency_key` in the tool's `params.arguments`. Server enforces via existing Redis SETNX.
- `query_*` tools — read-only; idempotency-by-construction.

Document the idempotency contract per-tool in the tool's `description` field.

§3.6 Observability — OTel propagation

MCP messages don't carry W3C TraceContext natively. Herald extension:

- **Streamable HTTP** — `traceparent` + `tracestate` HTTP headers propagate normally.
- **stdio** — Herald defines an MCP-extension parameter `_traceparent` injected into every `params` object. Documented in spec V3 §4x.x.

§3.7 Failure modes specific to Herald

1. **Tool description trust.** Per MCP §"Security #3", tool descriptions are untrusted unless from a trusted server. Herald is the server — but tool descriptions Herald accepts FROM external clients (e.g. when Herald consumes MCP) MUST be sanitized before being shown to humans.
2. **Sampling abuse.** If Herald requests sampling from the host (recursive LLM calls), it could be used to amplify cost. Herald MUST rate-limit `sampling/createMessage` invocations per tenant.
3. **Resource size.** A `resources/read` returning the full `events_processed` log could exceed gigabytes. Pagination + `range` parameters MUST be enforced.

§4. Step-by-step implementation guide for Herald

§4.1 Add the Go SDK as a submodule

```
# from repo root — DO NOT actually run; this is the plan
git submodule add git@github.com:modelcontextprotocol/go-sdk.git
        submodules/go-mcp-sdk
git -C submodules/go-mcp-sdk checkout v1.6.1
```

Per Helix §3, vendored SDKs go in as submodules, not `go get`. The `commons_mcp/` module imports via `replace` in its `go.mod`.

Alternative: spec V3 §9.1 carve-out allowing `go get` for upstream-stable SDKs (`@<version-tag>` pinning). Decision deferred to operator.

§4.2 New module `commons_mcp/`

New L1 module at `/commons_mcp/`. Structure (mirroring `commons_messaging/`):

```
commons_mcp/
├── go.mod
├── server/
│   ├── server.go          # MCP server wrapper that wires Herald tenants
│   ├── resources.go       # generic resource registration helpers
│   ├── tools.go           # generic tool registration helpers
│   ├── auth.go            # JWT extraction from OAuth bearer / stdio env
│   ├── otel.go            # W3C TraceContext extension
│   └── server_test.go
├── client/
│   ├── client.go          # MCP client wrapper for outbound calls
```

```
│   └── client_test.go
└── stdio_e2e_test.go      # real subprocess + real stdio assertion
```

§4.3 Wire MCP into every flavor's `serve` command

Add to `commons/cli/serve.go` (or a new `cli/serve_mcp.go`):

```
// SerHTML — Wave 4a: --mcp-stdio + --mcp-http flags wire MCP
        server
func ServeCmd(...) *cobra.Command {
    cmd := &cobra.Command{...}
    cmd.Flags().Bool("mcp-stdio", false, "serve MCP over stdio
        (exclusive with --mcp-http)")
    cmd.Flags().Bool("mcp-http", false, "serve MCP over
        Streamable HTTP at /v1/mcp")
    // ...
}
```

Per-flavor wiring: each flavor's `cmd/<flavor>/serve.go` registers its specific tools + resources via `commons_mcp/server.Register*`.

§4.4 Resources + tools per flavor

Flavor	Resources	Tools
pherald	`events://list`, `events://{id}`, `subscribers://list`	`notify`, `bulk_notify`
cherald	`compliance://posture`, `compliance://controls`, `compliance://attestations/{id}`	`query_compliance`, `attest`
sherald	`safety://state`, `safety://incidents`	`query_safety`
bherald	`budget://state`, `budget://overruns`	`query_budget`
rherald	`risk://register`, `risk://heatmap`	`query_risk`, `flag_risk`
iherald	`incident://timeline`, `incident://{id}`	`acknowledge_incident`, `resolve_incident`
scherald	`schedule://crons`, `schedule://upcoming`	`enqueue_job`, `cancel_job`

§4.5 New e2e_bluff_hunt invariants

Add to `scripts/e2e_bluff_hunt.sh`:

- **E48** — `mcp` SDK example client launches `pherald serve --mcp-stdio` as subprocess; sends `initialize`; asserts `protocolVersion: "2025-11-25"`.
- **E49** — client sends `tools/list`; asserts `notify` tool is present.
- **E50** — client invokes `tools/call notify` with a valid message + idempotency_key; asserts success.
- **E51** — second invocation with SAME idempotency_key returns the same result (idempotency proof).

- **E52** — `pherald serve --mcp-http` listens on `:7104/v1/mcp`; client connects via Streamable HTTP; asserts session-id handshake.
- **E53** — JWT-less request to `/v1/mcp` returns 401 (auth proof).
- **E54** — mutation: comment out `commons_mcp/server/server.go` Register() call; rebuild; E48 FAILS (mutation gate proves the wire is load-bearing).
- **E55** — concurrency: 50 concurrent MCP clients hit the same pherald; all succeed; no deadlock.

§4.6 Mutation gates

Per Helix §1.1, every claim-bearing test gets a paired mutation gate. Sketches:

- Mutation 1: remove the `tools/call notify` handler — E50 MUST FAIL.
- Mutation 2: remove the JWT middleware on `/v1/mcp` — E53 MUST FAIL.
- Mutation 3: remove Redis SETNX — E51 MUST FAIL.

Each mutation gate runs as its own subprocess + reverts at the end.

§4.7 Spec V3 amendments

Add to `docs/specs/mvp/specification.V3.md`:

- **§4x.0** — MCP wire format + transport choice (stdio + Streamable HTTP).
- **§4x.1** — auth: OAuth 2.0 + JWT layering.
- **§4x.2** — tenant_id propagation.
- **§4x.3** — idempotency injection per-tool.
- **§4x.4** — OTel `_traceparent` extension parameter.
- **§4x.5** — per-flavor resource + tool catalog.

Per §106, this spec edit triggers the implementation comprehensively in the same wave.

§4.8 HRD scaffolding

Open HRDs (suggested numbering, subject to operator):

- **HRD-200** — bootstrap `commons_mcp/` module + go.work entry.
- **HRD-201** — implement MCP server stdio transport.
- **HRD-202** — implement MCP server Streamable HTTP transport.
- **HRD-203** — implement MCP client wrapper.
- **HRD-204** — pherald MCP integration + resources/tools.
- **HRD-205** — cherald MCP integration.
- **HRD-206** — sherald MCP integration.
- **HRD-207** — bherald MCP integration.
- **HRD-208** — rherald MCP integration.
- **HRD-209** — iherald MCP integration.
- **HRD-210** — scherald MCP integration.
- **HRD-211** — JWT-over-OAuth-2.0 token issuance.
- **HRD-212** — idempotency contract per tool.
- **HRD-213** — OTel `_traceparent` extension.
- **HRD-214** — rate limit `sampling/createMessage`.
- **HRD-215** — pagination on `resources/read`.
- **HRD-216** — e2e_bluff_hunt E48–E55.
- **HRD-217** — mutation gate suite for MCP.
- **HRD-218** — spec V3 §4x amendments.

- **HRD-219** — operator credentials guide update (MCP section).
- **HRD-220** — close Wave 4a + bump CLAUDE.md/AGENTS.md.

§5. §107 anti-bluff testing strategy

The bar: the end-user (a developer running Claude Desktop / Cursor / Continue) can configure Herald as an MCP server in their LLM client's config, see Herald's tools/resources appear in the LLM's tool picker, and successfully invoke them with results that match Herald's REST surface byte-for-byte (mod transport framing).

§5.1 Test categories

Each test produces PHYSICAL evidence. No metadata-only PASS allowed.

§5.1.1 Happy-path tests

- **Test 1: stdio handshake.** Subprocess `pherald serve --mcp-stdio`; send `initialize` JSON-RPC; assert response contains `protocolVersion: "2025-11-25"` AND `serverInfo.name == "herald-pherald"`. **Physical proof: capture the subprocess stdout + diff against expected bytes.**
- **Test 2: tool invocation round-trip.** After handshake, send `tools/list` → assert `notify` present. Send `tools/call` with valid args → assert response. Then independently query Herald's REST `/v1/events/{eventID}` → assert the event landed identically. **Physical proof: same row in `events_processed` Postgres table whether invoked via REST or MCP.**
- **Test 3: Streamable HTTP session.** Start `pherald serve --mcp-http`; `curl -X POST` with `initialize` → assert `Mcp-Session-Id` response header. Use that ID for `tools/list`. **Physical proof: tcpdump on :7104 shows the session header round-tripping.**

§5.1.2 Edge cases

- **Test 4: oversized payload.** Send `tools/call notify` with a 10 MiB body → assert protocol-level rejection at the configured limit (e.g. 1 MiB) with JSON-RPC error code -32600.
- **Test 5: malformed JSON.** Send invalid UTF-8 → assert parse error -32700.
- **Test 6: slow client.** Open Streamable HTTP connection; pause mid-request for 30s → assert server-side timeout per `commons_tls`'s read-deadline.

§5.1.3 Failure modes

- **Test 7: auth fail.** Streamable HTTP without `Authorization: Bearer` → 401. With invalid signature → 401. With expired token → 401.
- **Test 8: capability mismatch.** Send `sampling/createMessage` to a server that didn't advertise `sampling` capability → JSON-RPC -32601 method not found.
- **Test 9: version skew.** Client `protocolVersion: "2024-11-05"` (old) → server responds with `protocolVersion: "2025-11-25"` (its supported version); client decides whether to disconnect.
- **Test 10: tenant isolation.** Tenant A's JWT calls `resources/read events://{id}` for an event owned by Tenant B → 403. **Physical proof: the resource read does NOT return Tenant B data.**

§5.1.4 Performance

- **Test 11: throughput.** 1000 concurrent `tools/call notify` invocations on Streamable HTTP → assert p50 < 50ms, p99 < 500ms.
- **Test 12: stdio latency.** stdio round-trip on a single tool call → assert < 10ms median (no network).

§5.1.5 Concurrency

- **Test 13: 50 concurrent clients.** 50 separate subprocesses each running `claude-code --mcp-server pherald serve --mcp-stdio`, each invoking 100 tool calls → assert no deadlock, all 5000 calls succeed.

§5.2 Mutation gates

Each happy-path test has a mutation pair that proves the test isn't a bluff:

- **Mutation A:** delete the `tools/call notify` handler dispatcher line → Test 2 MUST FAIL with method-not-found.
- **Mutation B:** remove the `Authorization` header parsing in `commons_mcp/server/auth.go` → Test 7 MUST FAIL (auth accepts anything).
- **Mutation C:** remove the tenant filter in `events://` resource handler → Test 10 MUST FAIL (cross-tenant leak).
- **Mutation D:** stub out the Redis SETNX call → second `notify` with same idempotency_key creates a second event row.

§5.3 Real-client integration test (the gold-standard physical proof)

A separate `scripts/e2e_mcp_real_claude_code.sh` boots `pherald serve --mcp-stdio` in a docker container, configures `claude-code` CLI with `~/.config/claude-code/mcp_servers.json` pointing at the container, runs `claude-code mcp list-tools`, and asserts `notify` appears in the human-readable output. **This is the §107 gold-standard: a real downstream LLM client really sees the tools.** Skipped in CI by default (requires Anthropic API key); operator runs it before tagging Wave 4a complete.

§5.4 Wire-level inspection

- **tcpdump** the Streamable HTTP traffic during E52; assert `traceparent` header present + correct format.
- **strace** the stdio subprocess during E48; assert reads on FD 0, writes on FD 1, no FD 2 noise on happy path.
- **diff** the JSON-RPC framing byte-by-byte against a `golden/` test corpus.

§5.5 Size + latency invariants

- MCP-over-Streamable-HTTP responses MUST be Brotli-compressed when `Accept-Encoding: br` is present; assert size reduction vs identity encoding.
- stdio framing per the spec is "newline-delimited"; assert no embedded newlines in payloads (escape via JSON string encoding).

§6. Open questions for operator

1. **Server-only or server + client?** Recommendation: server first, client in Wave 4a+1.
2. **OAuth 2.0 Authorization Server — Herald-native or delegated?** Herald can either implement RFC 6749 + RFC 7591 itself, or delegate to Auth0 / Keycloak / Cognito. The latter is way simpler operationally. Operator decides.
3. **stdio per-tenant or HTTP multi-tenant primary?** Recommendation: HTTP multi-tenant is the default; stdio is opt-in for single-tenant dev/desktop scenarios.
4. **Tool description sanitization policy?** What's the max length? Markdown allowed? HTML stripped? Set policy in spec V3 §4x.5.
5. **Sampling rate limit?** What's the per-tenant ceiling on `sampling/createMessage` calls/min? Suggest 60/min default, configurable.
6. **Vendor go-sdk via submodule or `go get`?** Helix §3 says submodule; operator can carve out a §9.1 exception. Recommend SUBMODULE for consistency.
7. **Resource pagination scheme?** MCP doesn't specify; we need to invent. Suggest `params.cursor` + `params.limit` per RFC 8288 link semantics.
8. **Should Herald's MCP server publish to the public registry at `github.com/modelcontextprotocol/servers`?** Marketing/visibility decision.

§7. References

(All fetched 2026-05-22.)

- **Spec.** Model Context Protocol Specification 2025-11-25 — https://modelcontextprotocol.io/specification/2025-11-25
- **Schema.** schema.ts source-of-truth — https://github.com/modelcontextprotocol/modelcontextprotocol/blob/main/schema/2025-11-25/schema.ts
- **Transports section.** https://modelcontextprotocol.io/specification/2025-11-25/basic/transports
- **Why SSE was deprecated.** https://blog.fka.dev/blog/2025-06-06-why-mcp-deprecated-sse-and-go-with-streamable-http/
- **Official Go SDK.** https://github.com/modelcontextprotocol/go-sdk — v1.6.1, 4.6k stars, Apache 2.0, maintained in collaboration with Google. Min Go version: 1.24+ (tested up to 1.26).
- **Community Go SDK alt 1.** https://github.com/mark3labs/mcp-go
- **Community Go SDK alt 2.** https://github.com/metoro-io/mcp-golang
- **Public servers registry.** https://github.com/modelcontextprotocol/servers
- **2026 implementation guide.** https://fast.io/resources/mcp-server-golang/
- **Auth0 deep-dive on Streamable HTTP + security.** https://auth0.com/blog/mcp-streamable-http/
- **Survey of agent interop protocols** — arXiv:2505.02279 (MCP/ACP/A2A/ANP comparison).

§8. Catalogue-check verdict

Per §11.4.74:

- **vasic-digital org search:** no module matching `mcp` or `model-context-protocol`. No-match.
- **HelixDevelopment org search:** no module matching `mcp` or `model-context-protocol`. No-match.

Verdict: no-match → vendor as Herald-internal submodule under `submodules/go-mcp-sdk/` pointing at `github.com/modelcontextprotocol/go-sdk@v1.6.1`. Alternative: spec V3 §9.1 `go get` carve-out (operator decides).

Once vendored, `commons_mcp/go.mod` adds:

```
replace github.com/modelcontextprotocol/go-sdk => ../submodules/go-mcp-sdk
```

Pin update cadence: quarterly review of upstream releases; bump when a CVE lands or a new spec version drops.